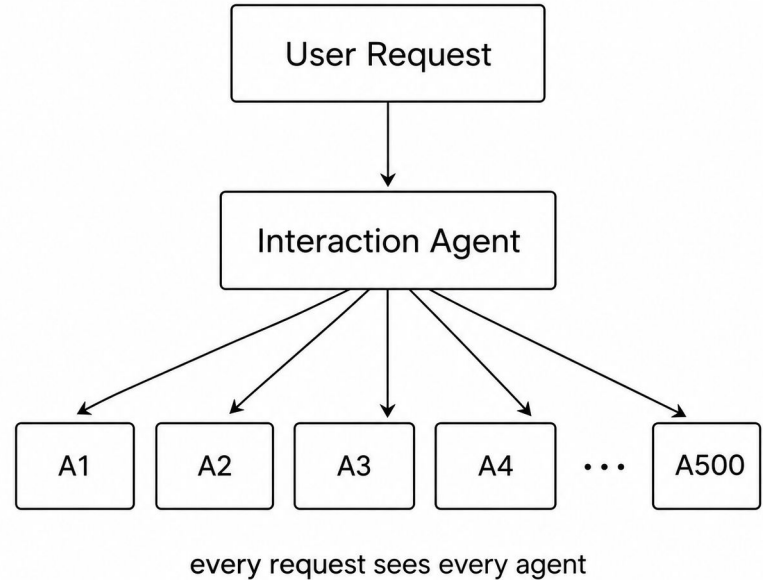


OpenPoke Re-Architecture

Suneru Perera

What's the Problem with Agent Overload?

OpenPoke has 500 agents. When you enter a message, the request is shown **every existing agent**, even when most are irrelevant.



Why is this bad?

More agents → more context → harder routing → more overload

Example From Code

In: `server/agents/interaction_agent/agent.py`

```
def _render_active_agents() -> str:
    roster = get_agent_roster()
    roster.load()
    agents = roster.get_agents()

    rendered: List[str] = []
    for agent_name in agents:
        name = escape(agent_name or "agent", quote=True)
        rendered.append(f'<agent name="{name}" />')

    return "\n".join(rendered)
```

```
sections.append(_render_conversation_history(transcript))
sections.append(
    f"<active_agents>\n{_render_active_agents()}\n</active_agents>"
)
sections.append(_render_current_turn(latest_text, message_type))

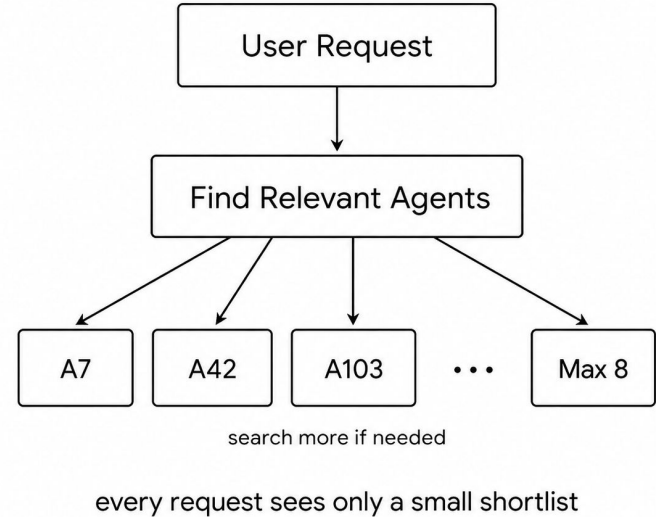
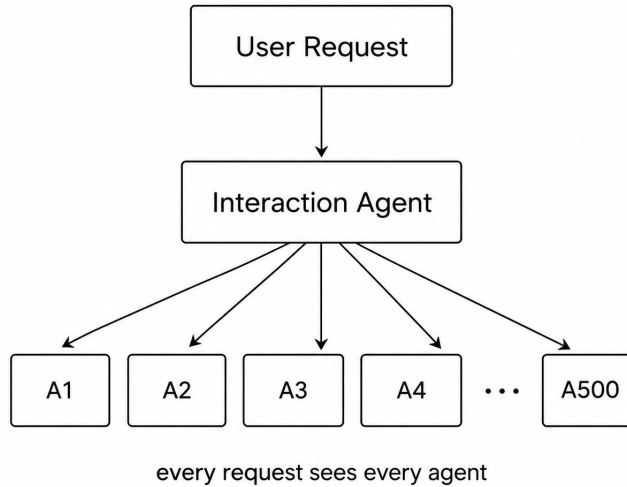
content = "\n\n".join(sections)
return [{"role": "user", "content": content}]
```

Get all agents → loop through them → format every one.

Put that full list into the prompt for this request.

Fix?

Instead of showing every agent, create a shortlist of the relevant ones.



Why?

Less context -> Fewer irrelevant agents in the prompt

Easier choice -> The model compares a small relevant set

Still flexible -> Search the full directory if the right agent isn't shown

How'd I test this?

- 1. Create synthetic agent rosters with realistic distractors**
- 2. Create example requests where we know the correct agent**
- 3. Repeat with 10, 100, and 500 agents**
- 4. Shuffle the extra agents**
- 5. Check if it still finds the right agent**
- 6. Measure how much context it uses**
- 7. Measure the recall per request**

Example of Synthetic Agent Roster Creation

```
+  
dict(id='opaque',owner='queue:806',project='Cobalt',contact  
='Bastien',duty='export declarations',request='Prepare the  
export declaration for Cobalt with Bastien.'),
```

```
47 + name = f'{project} {contact} {duty} / {i}' if i % 2
```

Create synthetic agents with known responsibilities, then generate up to 500 distractor agents with overlapping projects, contacts, and tasks. For each request, measure whether the correct agent still appears in the shortlist.

Context taken From 10 - 500 Agents

```
8 + "summary": [  
9 +   {  
10 +     "agents": 10,  
11 +     "executions": 35,  
12 +     "mean_roster_bytes": 501,  
13 +     "min_roster_bytes": 481,  
14 +     "max_roster_bytes": 527  
15 +   },  
16 +   {  
17 +     "agents": 100,  
18 +     "executions": 35,  
19 +     "mean_roster_bytes": 4905,  
20 +     "min_roster_bytes": 4869,  
21 +     "max_roster_bytes": 4940  
22 +   },  
23 +   {  
24 +     "agents": 500,  
25 +     "executions": 35,  
26 +     "mean_roster_bytes": 24964,  
27 +     "min_roster_bytes": 24908,  
28 +     "max_roster_bytes": 25019  
29 +   }  
30 + ],
```

Added more agents and measured how much context OpenPoke included in every request.

10 agents -> 501 B
100 agents -> 4.9 KB
500 agents -> 25.0 KB

50× more agents → ~50× more context

What changed in the Code

```
46 + def _render_active_agents(latest_text: str = "", transcript:
    str = "") -> str:
47 +     import json
48 +     block = get_agent_roster().shortlist(latest_text,
        transcript)
49 +     page = json.loads(block.split("\n", 1)[1].rsplit("\n", 1)
        [0])
50 +     emit("shortlist", candidates=page["candidates"],
        roster_bytes=len(block.encode("utf-8")))
51 +     return block
```

Instead of loading the full roster, we use a ranked shortlist.

```
18 + BUDGET, LIMIT = 4096, 8
```

Max 8 agents with 4096 bytes.

```
128 + def search_agents(query: str, cursor: Optional[str] = None) ->
    ToolResult:
129 +     page = get_agent_roster().search(query, cursor)
130 +     emit("search", query=query, candidates=page["candidates"],
        paginated=bool(cursor),
131 +         response_bytes=len(json.dumps(page,
        ensure_ascii=False, separators=(",", ":")).encode()))
132 +     return ToolResult(success=True, payload=page)
```

Search the full directory only when needed.

Results?

```
+ {  
+   "size": 10,  
+   "variant": "enriched",  
+   "unambiguous_cases": 25,  
+   "shortlist_recall": 1.0,  
+   "first_search_recall": 1.0,  
+   "max_roster_bytes": 2684,  
+   "mean_baseline_bytes": 501  
+ },  
+ {  
+   "size": 100,  
+   "variant": "enriched",  
+   "unambiguous_cases": 25,  
+   "shortlist_recall": 0.92,  
+   "first_search_recall": 0.92,  
+   "max_roster_bytes": 2938,  
+   "mean_baseline_bytes": 4905  
+ },  
+ {  
+   "size": 500,  
+   "variant": "enriched",  
+   "unambiguous_cases": 25,  
+   "shortlist_recall": 0.6,  
+   "first_search_recall": 0.6,  
+   "max_roster_bytes": 3083,  
+   "mean_baseline_bytes": 24964  
+ }
```

10 agents -> 2.6 KB

100 agents -> 2.9 KB

500 agents -> 3.0 KB

Overall ~ 88% context drop

But...

Retrieval Recall Dropped:

10 agents → 100%

100 agents → 92%

500 agents → 60%

Context stayed small, but finding the right agent in the shortlist got harder at scale.

Fix? Improve bounded retrieval with BM25 ranking

```
210         if enriched:
211             -         fields += [("entities", "
                ".join(r["entities"]), 3), ("purpose", r["purpose"], 2),
                ("recent", " ".join(r["recent_instructions"]), 1)]
212             -         score, why = 0, []
213             -         for label, text, weight in fields:
214                 -         matches = q & terms(text)
215                 -         score += weight * sum(weights[t] for t in
                matches)
216             -         score += weight * 2 * len(query_phrases &
                phrases(text))
```

```
235 +         for text in [r["purpose"],
                *r["recent_instructions"]]:
236 +             responsibility |=
                Counter(retrieval_tokens(text))
237 +             fields["responsibility"] = responsibility
238 +             documents.append(fields)
239 +         if not documents:
240 +             return []
241 +             weights = {"name": 1.0, "responsibility": 2.0}
242 +             averages = {field: sum(sum(d.get(field, {}).values())
                for d in documents) / len(documents)
243 +             for field in weights}
```

Switch from lexical matching to fieldaware BM25 ranking.

New Results...

Agents	V1 Recall	V2 Recall	Context
10	100%	100%	2.37 KB
100	25%	90%	2.38 KB
500	25%	86.7%	2.45 KB

```
125 + {
126 +   "size": 500,
127 +   "variant": "v2",
128 +   "eligible": 60,
129 +   "shortlist_recall": 0.8666666666666667,
130 +   "first_search_recall": 0.8666666666666667,
131 +   "bound_violations": 0,
132 +   "max_roster_bytes": 2453,
133 +   "misses_by_task": {
```

Now: 500 agents: 25% → 86.7% recall, Context stayed ~2.45 KB. W!!!!

Live Model Testing:

1. Create 2 fixed routing cases with a known correct agent.
2. Build a 500-agent roster with lots of distractors.
3. Run both Original OpenPoke and V2 using Claude Sonnet 4.
4. Compare whether they chose the right agent and how much context/tokens they used.

Live Model Results

Results in evals/results/live-final-v2-500.json

500 agents	Original	Final V2
Initial context	39.25 KB	17.96 KB
Prompt tokens	22.53k	14.48k
Correct routing	2/2	2/2
Failures	2	0

Same routing accuracy, with ~54% less initial context and ~36% fewer observed prompt tokens.